

Kirana Marketplace

20-Day Two-Person Build Plan — From Scratch to Advanced Level

Team: Varshith (Backend & Database) · Hasini (Frontend)

This plan takes the Multi-Vendor Hyperlocal E-Commerce Platform for Local Kirana Stores from an empty repository to a deployed, demo-ready application in 20 days, split into 5 phases. Each day lists Varshith's backend/database tasks and Hasini's frontend tasks side by side, plus an end-of-day integration checkpoint so the two tracks never drift apart. It includes the core customer/vendor/admin system, the handwritten-list ordering feature, payments, location, notifications, security hardening, and a set of advanced/AI features, plus suggestions to make the project stand out.

How to use this plan: work in 2 blocks/day (~3-4 hrs each). Do a 15-minute morning sync (what's blocking whom) and the listed end-of-day integration check. Use a shared Git repo with **backend/** and **frontend/** folders, feature branches, and daily merges to **develop**.

PHASE 1 (Days 1-4): Foundations & Core Catalog

Goal: project scaffolding, database schema, authentication for all 3 roles, and a browsable product catalog.

DAY 1 — Project Setup & Planning

Varshith (Backend & Database)	Hasini (Frontend)
• Create GitHub repo, branch strategy (main/develop/feature-*) • Set up Node.js + Express project skeleton, ESLint/Prettier • Install & configure PostgreSQL (local + a free cloud instance, e.g. Neon/Supabase) • Design full ER diagram: Users, Customers, Vendors, Admins, Stores, Products, Categories, Orders, ListOrders, etc. (use dbdiagram.io) • Write and run initial migrations for Users, Customers, Vendors, Admins tables	• Set up React (Vite) + Tailwind CSS project skeleton • Set up React Router with route stubs for Customer, Vendor, Admin areas • Create a shared component library folder (Button, Input, Card, Modal, Badge) • Wireframe the full site in Figma (or a quick paper/whiteboard sketch photographed) for Home, Product, Cart, Vendor Dashboard, Admin Dashboard • Agree with Varshith on the exact JSON shape for API responses (sample payloads)

End-of-day sync: Confirm the ER diagram and sample API payload shapes together — this contract drives both tracks for the rest of the project.

DAY 2 — Authentication (Customer + Vendor)

Varshith (Backend & Database)	Hasini (Frontend)
• Build auth APIs: register, login, logout, forgot/reset password (JWT + bcrypt) • Add OTP login endpoint (mock/stub SMS provider for now) • Add Google OAuth login endpoint • Add role-based middleware (customer / vendor / admin) guarding routes • Write Postman collection for all auth endpoints	• Build Login, Register, Forgot/Reset Password pages (Customer) • Build separate Vendor Login/Register pages • Add form validation (react-hook-form + zod/yup) • Set up auth state management (Context API or Zustand) + protected routes • Wire up API calls using the Postman collection as reference

End-of-day sync: Do a live test: register a customer and a vendor end-to-end through the UI hitting the real API.

DAY 3 — Product & Category Catalog (Backend)

Varshith (Backend & Database)	Hasini (Frontend)
• Create Products, Categories, Brands, Inventory tables + migrations • Build APIs: list/search/filter/sort products, get product by id, list categories • Add image upload endpoint (Cloudinary/AWS S3 integration) • Seed the database with realistic sample kirana products across 5+ categories • Add pagination to product listing endpoint	• Build Home page: categories, featured/popular products, offers banner (static for now) • Build Product Listing page with filters (price, brand, category, rating) and sorting • Build Product Details page (images, price, description, ratings, Add to Cart/Wishlist buttons) • Build reusable ProductCard and CategoryChip components • Apply the frontend-design skill: pick a distinct visual identity (colors/typography) instead of default Tailwind look

End-of-day sync: Connect the Product Listing and Details pages to the real seeded API data — confirm filters/sorting work against actual query params.

DAY 4 — Search, Location & Store Pages

Varshith (Backend & Database)

- Build product search API (text search across name/brand/category)
- Build Stores table + "nearby stores" API using pincode or lat/long radius query
- Build public Store Profile API (store info + its products)
- Add basic rate limiting and input validation middleware across all APIs so far
- Write integration tests for catalog + search endpoints (Jest + Supertest)

Hasini (Frontend)

- Build Search bar with live suggestions + Search Results page
- Build location selector (pincode entry / "use my location" with browser Geolocation API)
- Build public Store Profile page (banner, rating, hours, delivery radius, product grid)
- Polish Home page with real "Nearby Stores" section wired to the API
- Responsive pass: make Home, Listing, Product, Store pages mobile-friendly

End-of-day sync: Phase 1 demo: browse categories → search a product → view a store profile → see nearby stores, all on real data.

PHASE 2 (Days 5-9): Cart, Checkout, Orders & Vendor Operations

Goal: a customer can place a real multi-vendor order, and a vendor can manage it end to end.

DAY 5 — Cart & Wishlist

Varshith (Backend & Database)

- Build Cart, CartItems, Wishlist, WishlistItems tables
- Build APIs: add/remove/update cart item, get cart (grouped by vendor), move wishlist→cart
- Implement multi-vendor cart split logic (Option A: separate order per vendor — simplest for MVP)
- Add coupon validation API (basic: fixed code, min order, discount)
- Add server-side price/subtotal/delivery/discount calculation (never trust client totals)

Hasini (Frontend)

- Build Cart page grouped visually by vendor/store, with quantity controls
- Build Wishlist page with move-to-cart action
- Build coupon input + applied-coupon UI on cart
- Show live subtotal, delivery charge, discount, and final total from the API (not calculated client-side)
- Add empty-state illustrations for empty cart/wishlist

End-of-day sync: Add 3 products from 2 different vendors to the cart and confirm it correctly splits/labels items per vendor.

DAY 6 — Addresses & Checkout

Varshith (Backend & Database)

- Build Addresses table + CRUD APIs (Home/Work/Other, set default)
- Build Checkout API: validates stock, locks prices, creates Order(s) + OrderItems per vendor
- Add delivery option logic (standard/express/same-day/pickup) with basic charge rules
- Add order status enum + status-transition validation (no illegal jumps, e.g. Pending→Delivered)
- Write tests for the checkout flow, especially the multi-vendor order split

Hasini (Frontend)

- Build Address book UI (add/edit/delete/set default) with a simple map pin or manual lat-long entry
- Build Checkout page: address selection, delivery option, order summary, place-order button
- Build Order Confirmation page/screen
- Handle and display checkout errors gracefully (e.g. item out of stock at checkout)
- Add a loading/skeleton state pattern used across the app from here on

End-of-day sync: Full run-through: cart → address → delivery option → place order → confirmation screen, using real DB writes.

DAY 7 — Payments Integration

Varshith (Backend & Database)

- Integrate Razorpay (or Stripe) test mode: create order, verify payment signature, mark order paid
- Build Payments table + payment status webhook handler
- Implement Cash on Delivery as a payment method (no gateway call, just status=COD-pending)
- Add refund-initiation API stub (Admin/Vendor triggers, marks status Refunded)
- Secure payment endpoints (never trust amounts from the client — recompute server-side)

Hasini (Frontend)

- Build Payment method selection UI (UPI/Card/Netbanking/Wallet/COD)
- Integrate Razorpay/Stripe checkout widget on the frontend
- Build payment success/failure screens
- Show payment status on the Order Details page
- Add basic client-side error handling for payment failures with a retry option

End-of-day sync: Do a full test transaction in gateway test mode from checkout to payment confirmation to order creation.

DAY 8 — Customer Orders & Notifications

Varshith (Backend & Database)

- Build "My Orders" API (list + detail, with live status)
- Build Notifications table + APIs (order placed/accepted/packed/delivered, payment events)
- Wire notification creation into every relevant status change (order, payment, refund)
- Add email notification (e.g. via SendGrid/Nodemailer) for order placed/delivered
- Add invoice generation API (simple PDF or HTML invoice per order)

Hasini (Frontend)

- Build "My Orders" list + Order Details page with a visual status tracker (stepper)
- Build in-app Notifications panel/bell icon with unread badge
- Build downloadable/viewable Invoice UI
- Add Ratings & Reviews UI on delivered orders (stars + text, product and store)
- Polish: toast notifications for key actions (added to cart, order placed, etc.)

End-of-day sync: Place an order, move it through statuses from the vendor side (see Day 9), and confirm the customer sees live status + a notification for each change.

DAY 9 — Vendor Dashboard, Inventory & Order Management

Varshith (Backend & Database)

- Build Vendor Dashboard summary API (today's sales/orders, product count, low stock, pending orders)
- Build Vendor Product Management APIs (CRUD, stock update, enable/disable, discount)
- Build automatic stock-status logic (Available/Low Stock/Out of Stock) + low-stock alert notification
- Build Vendor Order Management APIs: list new orders, accept/reject, update status through the pipeline
- Build Vendor Sales Dashboard API (daily/weekly/monthly sales, best/low sellers, average order value)

Hasini (Frontend)

- Build Vendor Dashboard home with summary cards
- Build Vendor Product Management UI (add/edit product form, image upload, stock & discount controls)
- Build Vendor Inventory table view with color-coded stock status
- Build Vendor Order Management UI: incoming orders with Accept/Reject, status-update buttons
- Build Vendor Sales Dashboard with charts (Recharts) for sales trends and best-sellers

End-of-day sync: Phase 2 demo: place a full order as a customer, accept and progress it through statuses as the vendor, and confirm dashboards update correctly.

PHASE 3 (Days 10-13): Admin Panel & the Handwritten-List Feature

DAY 10 — Handwritten List Upload Feature

Varshith (Backend & Database)

- Create ListOrders table (customer_id, vendor_id, image_url(s), note, status, converted_order_id)
- Build image upload API for list photos (multi-image support) to Cloudinary/S3
- Build APIs: submit list order, vendor views list orders, vendor transcribes items into a priced draft order, customer confirms → converts into a normal Order
- Trigger a real-time notification (WebSocket or push) to the vendor the moment a list is submitted
- Add status flow: Submitted → Viewed → Priced → Awaiting Confirmation → Confirmed

Hasini (Frontend)

- Build "Upload Handwritten List" flow on a store page: camera/gallery capture, multi-photo, optional note, address + delivery slot selection
- Build customer-side List Order tracking card in My Orders
- Build Vendor "List Orders" inbox with unread badge and full-screen zoomable photo viewer
- Build the vendor transcription UI: type item name → auto-suggest from catalog → add qty/price, mark items unavailable/substituted
- Build the customer confirmation screen for the priced draft before it becomes a real order

End-of-day sync: Do a real end-to-end test: photograph an actual handwritten list, upload it, get notified as the vendor, transcribe it, and confirm it as the customer.

DAY 11 — Admin Panel — Users & Catalog Control

Varshith (Backend & Database)

- Build Admin auth (separate role, seeded manually — no public admin signup)
- Build Admin Dashboard summary API (total customers/vendors/products, today's orders/revenue)
- Build Admin User Management APIs (view/block/unblock customers, approve/reject/suspend vendors)
- Build Admin Product/Category Management APIs (approve/reject/remove products, manage categories/brands)
- Enforce vendor approval gate: new vendors cannot list products until Admin-approved

Hasini (Frontend)

- Build Admin Login + Dashboard layout (separate app shell/sidebar from customer/vendor)
- Build Admin Dashboard home with summary widgets
- Build Customer & Vendor management tables with block/approve/suspend actions
- Build Category/Brand management UI (add/edit/delete, nested subcategories)
- Build a Vendor Approval queue view for pending vendor signups

End-of-day sync: Register a new vendor, confirm it's blocked from listing products until approved via the Admin panel, then confirm it can list after approval.

DAY 12 — Admin — Coupons, Offers & Analytics

Varshith (Backend & Database)

- Build Coupon Management APIs (platform-wide + vendor-specific, min order/discount rules)
- Build Offers/Promotions APIs (banner content, active date ranges)
- Build Analytics APIs: sales (daily/weekly/monthly/yearly), customer analytics (new/returning/active), vendor analytics (top vendors/revenue), product analytics (best-selling/most viewed)
- Add basic caching (in-memory or Redis) for expensive analytics queries
- Write tests for coupon validation edge cases (expired, below min order, max discount cap)

Hasini (Frontend)

- Build Coupon management UI (create/edit/list coupons)
- Build homepage Banner/Offers management UI for admin
- Build Analytics dashboards with charts: sales over time, top vendors, top products, customer growth
- Build Returns & Refunds management UI for admin (approve/reject/process)
- General UI polish pass across all Admin screens for consistency

End-of-day sync: Phase 3 demo: full admin walkthrough — approve a vendor, create a coupon, view analytics — plus the handwritten-list feature end to end.

DAY 13 — Returns, Refunds & Security Hardening

Varshith (Backend & Database)

- Build Returns/Refunds tables + APIs (request → approved → picked up → refund processed)
- Full security pass: helmet, CORS config, rate limiting, input sanitization on every endpoint
- Verify role-based authorization on every route (customer/vendor/admin cannot access each other's APIs)
- Add centralized error-handling middleware with consistent API error shape
- Review and fix any SQL injection / XSS risk points; enable HTTPS-only config for deployment

Hasini (Frontend)

- Build Return/Refund request UI for customers (from Order Details)
- Build Return status tracker UI
- Add global error boundary + a friendly 404/500 page
- Add form-level and network-error handling consistently across the app
- Accessibility pass: alt text, color contrast, keyboard navigation on key flows

End-of-day sync: Try to access a vendor-only API as a logged-in customer (and vice versa) and confirm it's correctly rejected — do this together as a mini security test.

PHASE 4 (Days 14-17): Advanced Features & Polish

DAY 14 — Notifications Channels & Real-Time Updates

Varshith (Backend & Database)

- Integrate SMS (e.g. Twilio/MSG91) and WhatsApp notification for key events
- Add WebSocket (Socket.io) support for real-time order-status and list-order notifications
- Add push notification support (web push) as a stretch goal
- Optimize database queries flagged as slow (add indexes on frequently filtered columns)
- Set up basic API documentation with Swagger/OpenAPI

Hasini (Frontend)

- Wire real-time order status updates via WebSocket (no manual refresh needed)
- Wire real-time "New List Order" alert on the Vendor dashboard
- Add a Notification Preferences screen (email/SMS/WhatsApp toggle) for customers
- Performance pass: lazy-load images, code-split routes, check Lighthouse score
- Cross-browser and mobile-device testing pass

End-of-day sync: Confirm a vendor sees a new order/list-order appear live without refreshing, from a second browser/customer session.

DAY 15 — AI Recommendations & Smart Features

Varshith (Backend & Database)

- Build a simple "frequently bought together" API (co-occurrence from OrderItems)
- Build a "recommended for you" API based on the customer's past orders/categories
- Build a basic demand-prediction stub (e.g. moving average of recent sales per product)
- Add barcode field lookup API (scan barcode → return matching product)
- Optional: integrate an OCR service (Google Vision/Tesseract) to pre-fill handwritten-list transcription

Hasini (Frontend)

- Add "Frequently Bought Together" section on Product Details page
- Add "Recommended for You" section on the Home page for logged-in customers
- Add a barcode-scanner UI option in search (using the device camera)
- Add a low-stock/demand insight widget on the Vendor Dashboard
- Add a voice-search button (Web Speech API) on the search bar as a stretch feature

End-of-day sync: Confirm recommendation sections show sensible results using the seeded order history — adjust seed data if results look empty or random.

DAY 16 — Testing & Bug Fixing

Varshith (Backend & Database)

- Write/expand backend test suite (Jest + Supertest) for all major flows: auth, catalog, cart, checkout, orders, list orders, admin actions
- Load-test key endpoints (e.g. product listing, checkout) with a simple script (k6/autocannon)
- Fix all bugs surfaced by test runs; check DB constraints/foreign keys are correctly enforced
- Review and finalize environment variable configuration (.env.example) for all environments
- Prepare seed/demo data script that resets the DB to a clean, impressive demo state

Hasini (Frontend)

- Write frontend tests for key components/flows (React Testing Library + Vitest/Jest)
- Manual QA pass across every screen on both desktop and mobile viewports
- Fix all UI bugs, broken states, and console warnings/errors
- Verify every loading, empty, and error state actually looks good (not a blank white screen)
- Prepare a short demo script (which screens/flows to click through) for the final walkthrough

End-of-day sync: Run the full demo script together end to end at least once before deployment — fix anything that breaks.

DAY 17 — Deployment

Varshith (Backend & Database)

- Deploy backend (Render/Railway) with environment variables and production DB (Neon/Supabase/AWS RDS)
- Set up production database migrations and run the seed script for demo data
- Configure HTTPS, CORS for the production frontend domain, and production payment gateway keys (test mode)
- Set up basic monitoring/logging (e.g. simple request logger + error logging service like Sentry)
- Set up a CI pipeline (GitHub Actions) to run tests on every push

Hasini (Frontend)

- Deploy frontend (Vercel) pointing at the production API URL
- Verify all environment-specific config (API base URL, gateway public key) is correct in production
- Run the full demo script again against the live deployed URLs
- Fix any deployment-specific issues (CORS, mixed content, broken image URLs)
- Add a custom domain (optional) and a favicon/branding pass

End-of-day sync: Full smoke test on the live production URLs: register, browse, order, pay (test mode), and vendor-fulfill an order.

PHASE 5 (Days 18-20): Documentation, Presentation & Final Polish

DAY 18 — Documentation

Varshith (Backend & Database)

- Write backend README: setup instructions, .env variables, how to run migrations/seed, API overview
- Generate/finalize Swagger API documentation and export a Postman collection for the whole API
- Document the database schema (ER diagram export) and architecture decisions (e.g. why PostgreSQL, why separate ListOrders flow)
- Write a short "known limitations & future work" section
- Tag a v1.0 release in Git with a clean changelog

Hasini (Frontend)

- Write frontend README: setup instructions, folder structure, key libraries used
- Take clean screenshots/GIFs of every major screen for the report/presentation
- Write a short user guide (customer, vendor, admin) with screenshots
- Clean up unused code, console.logs, and commented-out blocks
- Finalize consistent branding (logo, color palette, favicon) across the app

End-of-day sync: Cross-review each other's README by actually following it on a clean machine/branch to confirm it works.

DAY 19 — Presentation & Report Prep

Varshith (Backend & Database)

- Prepare architecture diagram slide (client → API → DB/payments/maps) and explain design decisions
- Prepare a slide/section on security measures taken (auth, role isolation, validation)
- Prepare answers for likely technical questions (why this DB, how multi-vendor cart splits work, how the handwritten-list feature works end to end)
- Record a backup demo video of key backend-driven flows (checkout, vendor order handling) in case of live-demo issues
- Finalize the project report technical sections (architecture, database, security, API list)

Hasini (Frontend)

- Build/finalize presentation slides (problem statement, features, tech stack, screenshots, demo)
- Record a backup demo video of the full customer/vendor journey including the handwritten-list feature
- Rehearse the live demo end to end at least twice, timed
- Prepare the project report UI/UX and feature-list sections with screenshots
- Prepare a 1-page feature summary handout/README badge for quick reviewer reference

End-of-day sync: Do one full joint dry run of the presentation + live demo together, timed, and fix any rough transitions.

DAY 20 — Final Buffer, Polish & Submission

Varshith (Backend & Database)

- Fix any last-minute bugs found during rehearsal; freeze the backend codebase
- Double-check production environment stability (restart services, re-run seed if needed)
- Final security review pass (no exposed secrets/keys in the repo, .env excluded via .gitignore)
- Prepare final submission package: repo link, deployed API URL, Postman collection, ER diagram
- Tag final release and back up the database

Hasini (Frontend)

- Fix any last-minute UI bugs found during rehearsal; freeze the frontend codebase
- Final responsive/browser check on the live deployed site
- Prepare final submission package: deployed site URL, screenshots, demo video, presentation deck
- Proofread the report and slides for typos and consistency
- Celebrate — the project is complete and demo-ready! ■

End-of-day sync: Final joint checklist: both live URLs work, demo video plays, report and slides are complete, repo is clean and pushed.

Suggestions to Make the Project Stand Out

Process & collaboration

- Use a shared Kanban board (GitHub Projects/Trello) with columns To Do / In Progress / Review / Done, one card per task in this plan — makes progress visible to both of you and to evaluators
- Daily 10-15 min sync (even a voice call) at the start, using the "End-of-day sync" line for the previous day as the agenda
- Agree on API contracts (request/response JSON shape) *before* building a feature, not after — this is the single biggest way two people working in parallel avoid rework
- Use conventional commit messages (feat:, fix:, docs:) and small, frequent commits/PRs rather than one giant commit per phase

Technical quality

- Consider TypeScript on the frontend (and optionally backend) — catches bugs early and reads as more professional in a portfolio/academic review
- Add request validation with a schema library (zod/Joi) on every backend endpoint, not just forms
- Add an API rate limiter (express-rate-limit) and helmet by default — small effort, strong security story
- Use environment-based config (.env.development / .env.production) from day one, never hard-code URLs/keys
- Add automated tests early rather than only in Phase 5 — even 1-2 tests per feature as you build it compounds into strong coverage by Day 16

Product & UX

- Design a distinct visual identity for the brand (a name, logo, and 2-3 signature colors) rather than default Tailwind blue/gray — this alone makes the demo far more memorable
- Add real, good-quality product photos and realistic Indian kirana product data (brand names, prices in ₹) rather than "Product 1, Product 2" placeholders — massively improves how polished the demo feels
- Add empty/loading/error states everywhere — reviewers notice broken-looking blank screens more than missing features
- The handwritten-list feature is your strongest differentiator versus a generic e-commerce clone — give it its own polished screen and lead with it in the demo
- Consider a short onboarding tooltip tour for first-time vendors ("add your first product") to show product-thinking, not just CRUD

Features worth adding if time allows

- Multi-language support (English/Telugu/Hindi) for a hyperlocal Indian grocery app — strong realism boost
- A simple delivery-partner role/view (even just "mark as picked up / delivered" by a delivery person) to show you understand the full logistics chain, not only customer↔vendor
- Dark mode toggle — small effort, visibly polished
- "Repeat last order" button on the customer's order history for realistic grocery-reordering behavior
- A subscription/recurring-order feature (e.g. "deliver milk every day at 7 AM") — common real kirana use case and a great advanced-feature talking point

For the demo/evaluation

- Open the demo with the problem statement in one sentence, then go straight to the handwritten-list upload — it's the feature no generic tutorial project has

- Show the same order from both sides live (customer places it in one window, vendor accepts/packs it in another) to visually prove the real-time flow works
- Have the architecture diagram and ER diagram ready as a slide — evaluators often ask "why this database/stack" and a clear diagram answers it instantly
- Mention the security measures explicitly (role isolation, JWT, validation) even if not asked — it signals engineering maturity beyond just "it works"